CUDA Stream 与 CUDA Event

Introduction

由于 CPU 提交到 GPU 中的任务是异步执行的，为了控制异步之间的并发顺序，CUDA 引入了 stream 和 event 的概念。本文尝试分析理解 stream 和 event 的含义，因为充分理解和使用 stream 和 event 是正确、高效利用 GPU 的必要条件。

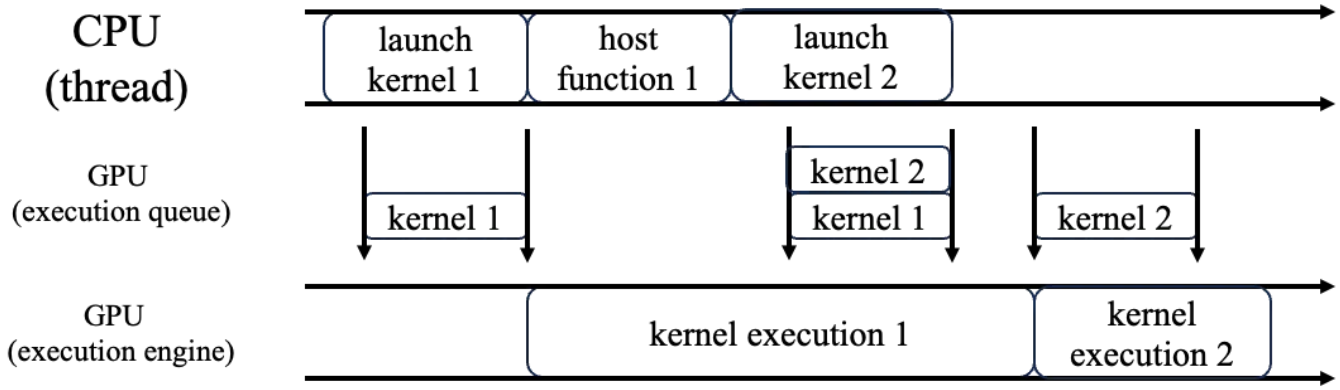
控制 Kernel 的并发执行顺序: CUDA Stream

只有 CPU 执行的情况

当不考虑 GPU 的时候，CPU 线程就在不断地执行指令。从时间维度上看，就是这样的一条线：

GPU 只有一个执行单元

GPU 相当于 CPU 的附属硬件，当 GPU 有一个执行单元时，由 CPU 下发任务给 GPU，从时间维度上看，CPU 上的任务和 GPU 上的任务是**异步的**。



上图中在 CPU 和 GPU 之间增加了一个 GPU 队列，以实现二者执行任务的异步性。当我们在 CPU 上调用 `kernelLaunch` 时，本质就是把这个 kernel 放入到 GPU 的**执行队列**

(execution queue) 中。然后，驱动负责维护这个队列，每当**执行引擎 (execution engine)** 空闲时，就执行队列中的 kenel。

早期的 GPU 硬件上只有一个 exection engine，因此，无论是哪个进程或哪个线程发起的 kernelLaunch 都会在同一个队列中排队，这可以看作是 CUDA 中的 `legacy default stream` 。

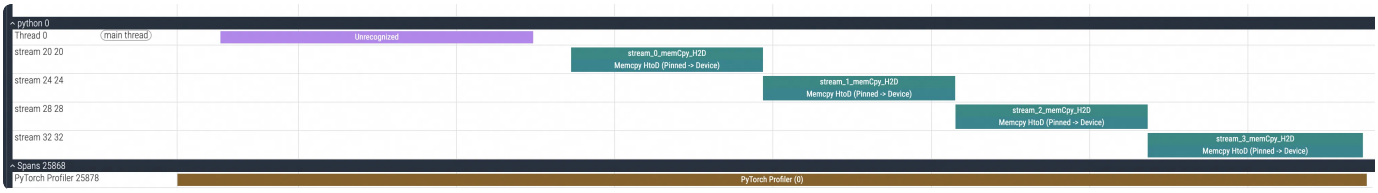
随着 GPU 的发展，出现了多个 `execution engine` ，首先出现的是 `kernel execution engine` 与 `memory operation engine` ，也就是 execution engine 从 1 个提高到了 2 个，分别负责 cuda kernel execution 与 memory operation (例如 memory copy)。多出来的 engine 就叫 `asyncEngine` ，因为它可以和主要负责执行的 engine 之间异步执行。

asyncEngine memory 测试输出 Bash

```bash
async engine数量: 1
已创建 4 个 CUDA stream
--------------------------------------------------------------------------------
各Stream的Tensor配置:
  Stream 0: shape=(4096, 4096), dtype=torch.float32, 元素数=16,777,216, 大小=64.00 MB
  Stream 1: shape=(4096, 4096), dtype=torch.float32, 元素数=16,777,216, 大小=64.00 MB
  Stream 2: shape=(4096, 4096), dtype=torch.float32, 元素数=16,777,216, 大小=64.00 MB
  Stream 3: shape=(4096, 4096), dtype=torch.float32, 元素数=16,777,216, 大小=64.00 MB
--------------------------------------------------------------------------------
执行完成!
总耗时: 1879.40 ms
创建了 4 个GPU tensor
--------------------------------------------------------------------------------
Profiler trace已导出到: async_engine_trace_memory_ops.json

Profiler摘要:
-----------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
                                Name    Self CPU %      Self CPU   CPU total %     CPU total  CPU time avg     Self CUDA   Self CUDA %    CUDA total  CUDA time avg    # of Calls
-----------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
    Memcpy HtoD (Pinned -> Device)         0.00%       0.000us         0.00%       0.000us       0.000us       4.996ms       100.00%       4.996ms       1.249ms             4
                stream_3_memCpy_H2D         0.00%       0.000us         0.00%       0.000us       0.000us       1.359ms        27.20%       1.359ms       1.359ms             1
                stream_1_memCpy_H2D         0.00%       0.000us         0.00%       0.000us       0.000us       1.213ms        24.28%       1.213ms       1.213ms             1
                stream_2_memCpy_H2D         0.00%       0.000us         0.00%       0.000us       0.000us       1.213ms        24.27%       1.213ms       1.213ms             1
```

```
                 stream_0_memCpy_H2D         0.00%       0.000us         0.00%       0.000us       0.000us       1.211ms        24.25%       1.211ms       1.211ms             1
             multi_stream_memory_ops         8.25%     607.896us        48.72%       3.592ms       3.592ms       0.000us         0.00%       0.000us       0.000us             1
                        Unrecognized        26.80%       1.976ms        26.80%       1.976ms       1.976ms       0.000us         0.00%       0.000us       0.000us             1
                 stream_0_memCpy_H2D        11.46%     844.665us        11.94%     880.033us     880.033us       0.000us         0.00%       0.000us       0.000us             1
                     cudaMemcpyAsync         0.85%      62.810us         0.85%      62.810us      15.702us       0.000us         0.00%       0.000us       0.000us             4
                 stream_1_memCpy_H2D         0.63%      46.189us         0.81%      59.547us      59.547us       0.000us         0.00%       0.000us       0.000us             1
                 stream_2_memCpy_H2D         0.29%      21.055us         0.35%      25.963us      25.963us       0.000us         0.00%       0.000us       0.000us             1
                 stream_3_memCpy_H2D         0.45%      33.438us         0.58%      42.614us      42.614us       0.000us         0.00%       0.000us       0.000us             1
               cudaDeviceSynchronize        51.28%       3.781ms        51.28%       3.781ms       3.781ms       0.000us         0.00%       0.000us       0.000us             1
---------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
Self CPU time total: 7.373ms
Self CUDA time total: 4.996ms

-------------------------------------------------------------------------------------------------------------------------
```

📎 async_engine_trace_memory_ops.json



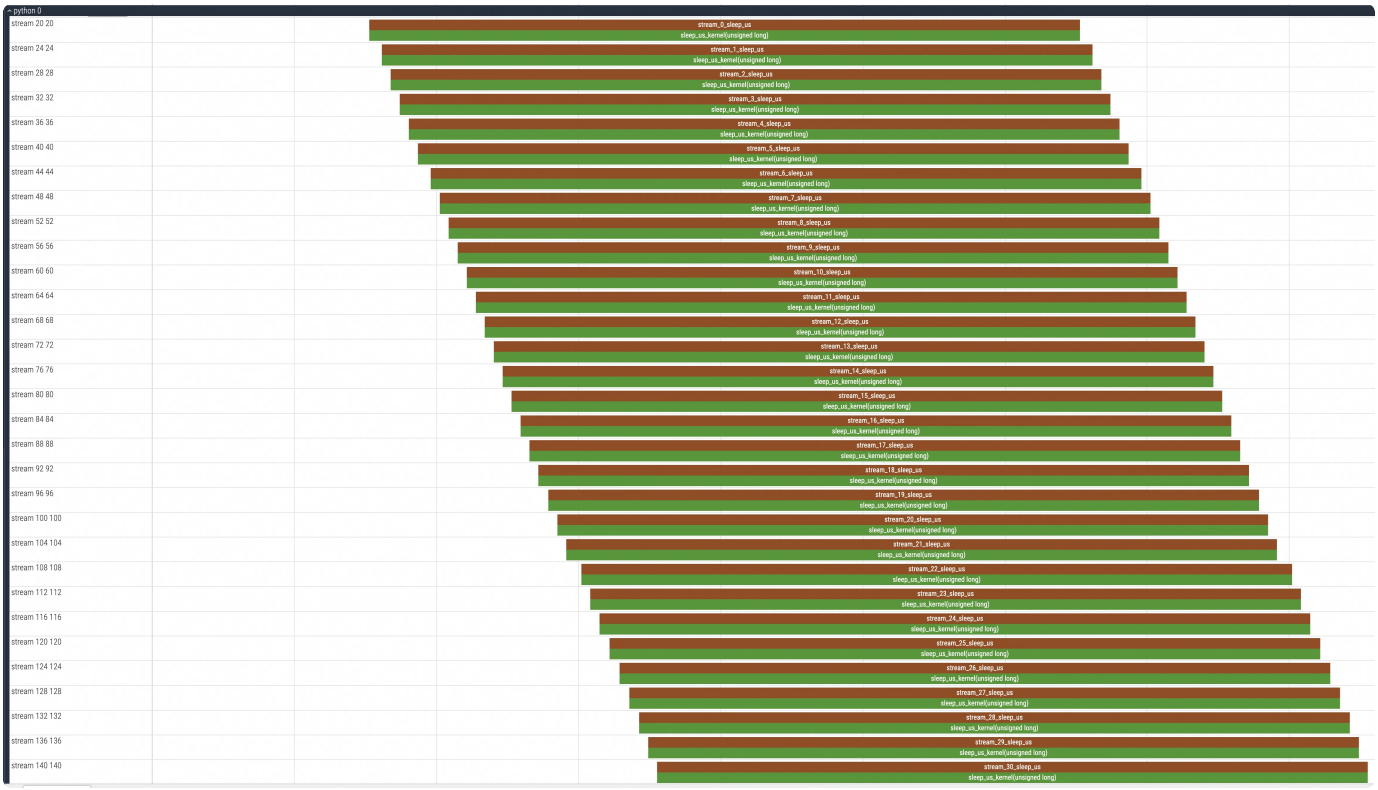
asyncEngine kernel 测试输出 Bash

```bash
async engine数量: 1
执行完成!
总耗时: 533.99 ms
--------------------------------------------------------------------------------
Profiler trace已导出到: async_engine_trace_sleep_us.json

Profiler摘要:
----------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
                                Name    Self CPU %      Self CPU   CPU total %     CPU total  CPU time avg     Self CUDA   Self CUDA %    CUDA total  CUDA time avg    # of Calls
----------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
    sleep_us_kernel(unsigned long)         0.00%       0.000us         0.00%       0.000us       0.000us        3.200s       100.00%        3.200s     100.001ms            32
                  stream_0_sleep_us         0.00%       0.000us         0.00%       0.000us       0.000us     100.002ms         3.13%     100.002ms     100.002ms             1
                 stream_21_sleep_us         0.00%       0.000us         0.00%       0.000us       0.000us     100.001ms         3.13%     100.001ms     100.001ms             1
                 stream_28_sleep_us         0.00%       0.000us         0.00%       0.000us       0.000us     100.001ms         3.13%     100.001ms     100.001ms             1
                  stream_7_sleep_us         0.00%       0.000us         0.00%       0.000us       0.000us     100.001ms         3.13%     100.001ms     100.001ms             1
                 stream_14_sleep_us         0.00%       0.000us         0.00%       0.000us       0.000us     100.001ms         3.13%     100.001ms     100.001ms             1
                  stream_5_sleep_us         0.00%       0.000us         0.00%       0.000us       0.000us     100.001ms         3.13%     100.001ms     100.001ms             1
                 stream_18_sleep_us         0.00%       0.000us         0.00%       0.000us       0.000us     100.001ms         3.13%     100.001ms     100.001ms             1
                 stream_19_sleep_us         0.00%       0.000us         0.00%       0.000us       0.000us     100.001ms         3.13%     100.001ms     100.001ms             1
```

```
                  stream_20_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_31_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_11_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_17_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                   stream_1_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                   stream_2_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                   stream_4_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_10_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_22_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_16_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_26_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_30_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_12_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_13_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                   stream_3_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
   100.001ms             1
                  stream_23_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
```

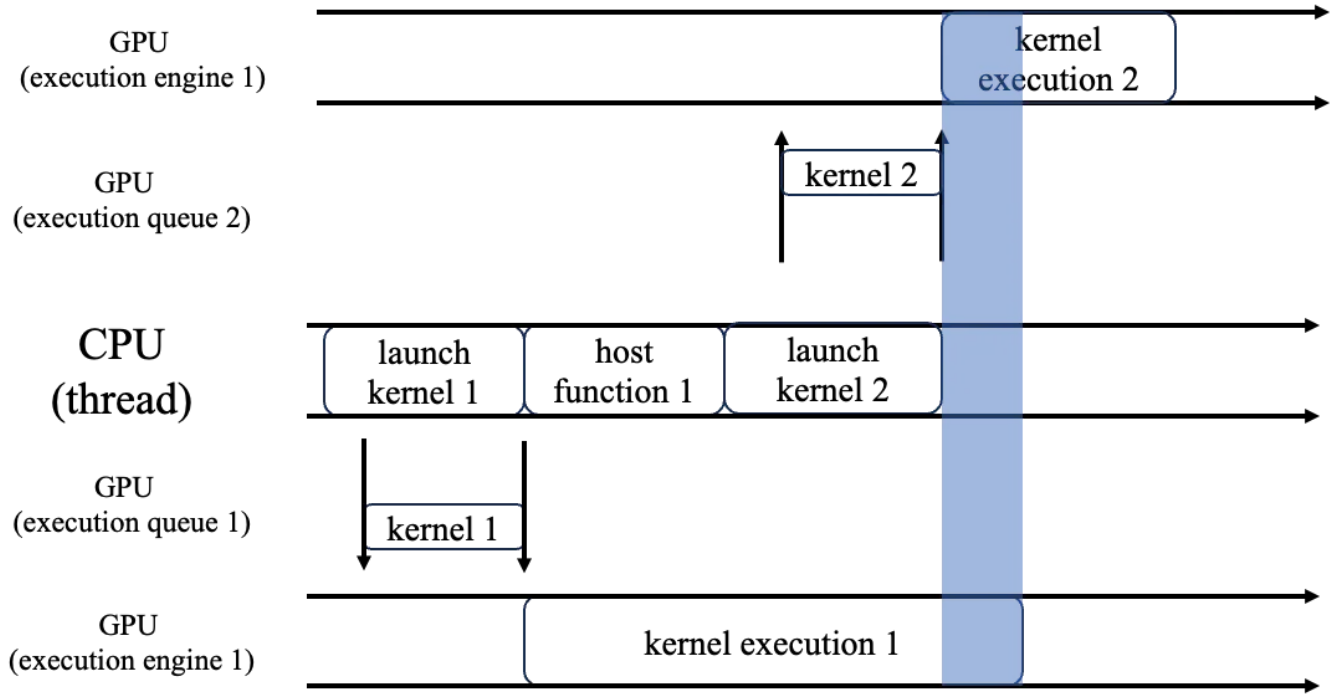
```
    100.001ms             1
                  stream_27_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
    100.001ms             1
                   stream_8_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
    100.001ms             1
                   stream_9_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
    100.001ms             1
                  stream_15_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
    100.001ms             1
                  stream_25_sleep_us         0.00%       0.000us         0.0
0%       0.000us       0.000us     100.001ms         3.12%     100.001ms
    100.001ms             1
-----------------------------------  -------------  -------------  -----------
--  -------------  -------------  -------------  -------------  -------------
-------------  -------------
Self CPU time total: 149.114ms
Self CUDA time total: 3.200s

-------------------------------------------------------------------------------
```

📎 [async_engine_trace_sleep_us.json]



GPU 有 2 个执行单元

当有两个或多个 GPU 执行单元时，可以让 GPU kernel 之间并行起来：



上图中蓝色阴影区域就是两个 GPU kernel 并发执行的时间段。值得注意的是，上图中的 kernel1 和 kernel2 **是否能并发执行是由用户决定的**，硬件不能擅作主张，否则万一 kernel2 要读取 kernel1 算出来的数据，那并发执行的结果就是错的。为了给用户提供这种控制权，于是就有了 stream 的概念。**一个 stream 就对应于一个执行队列**，用户可以自行决定是否把两个 kernel 分开放在两个队列里。

> stream 是 cuda 为上层应用提供的抽象，应用可以创建任意多个 stream，下发任意多个 kernel。但如果正在执行的 kernel 数目超过了硬件的 execution engine 的数量，那么即使当前 stream 里没有 kernel 正在执行，下发的 kernel 也必须在队列里继续等待，直到有硬件资源可以执行。

总结：**一个 GPU kernel 可以执行的必要条件是它在 stream 的队列首位且存在执行 kernel 的空闲硬件资源。**

创建 stream 的时候，我们也可以用 `cudaStreamCreateWithPriority` 为它指定优先级。当多个 GPU kernel 都可以执行的时候，cuda driver 负责调度，优先执行 high priority 的 stream 里的 kernel。

创建 stream 指定优先级 C++

```cpp
cudaStream_t stream1, stream2;
int          priorityHigh, priorityLow;
cudaDeviceGetStreamPriorityRange(&priorityLow, &priorityHigh);
cudaStreamCreateWithFlags(&stream1, cudaStreamDefault);
cudaStreamCreateWithPriority(&stream2, cudaStreamDefault, priorityHigh);
```

stream 的常用操作

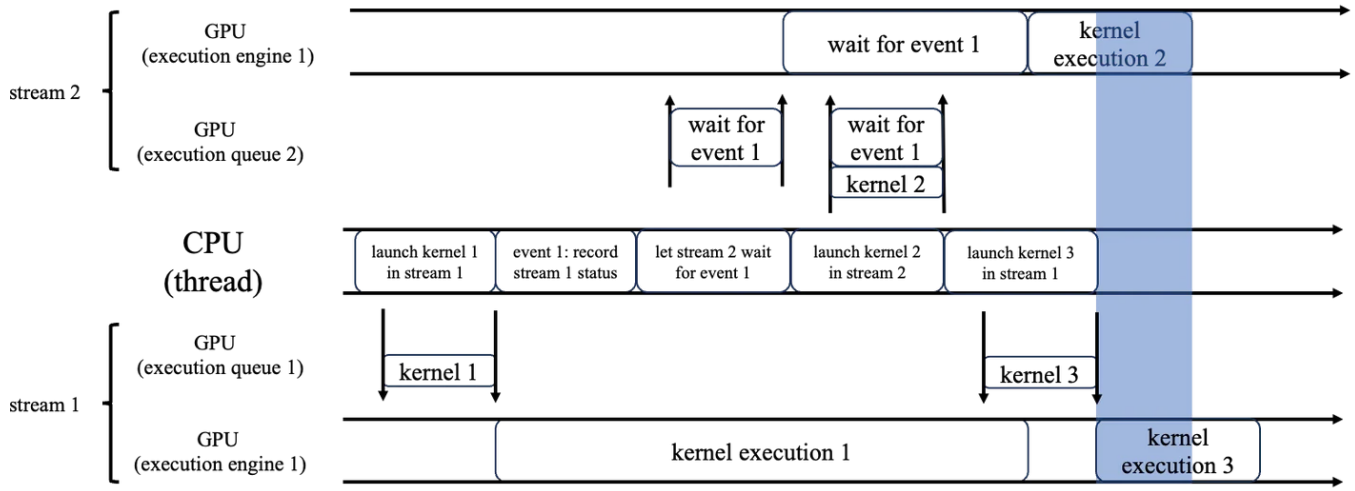
不仅可以从 CPU thread 对 stream下发 kernel (最常见的 stream 操作)，还可以：

- 查询 stream 里的队列是否为空，即 `cudaStreamQuery`
- 等待直到 stream 里的队列为空，即 `cudaStreamSynchronize`

CUDA Event

CUDA Event 控制 stream 之间的依赖

不仅可以从 CPU thread 操作 stream，也可以在一个 stream 上面操作另一个 stream，具体的操作就是**一个 stream 上面的 kernel 想等待另一个 stream 上面的 kernel 执行结束之后再执行**。为了实现这个目标，需要记录另一个 stream 上面的队列状态，这就是 `CUDA Event` 。让我们来看一个具体的例子：



它实现的功能是：

- 在 stream 1 上面执行 kernel 1 和 kernel 3

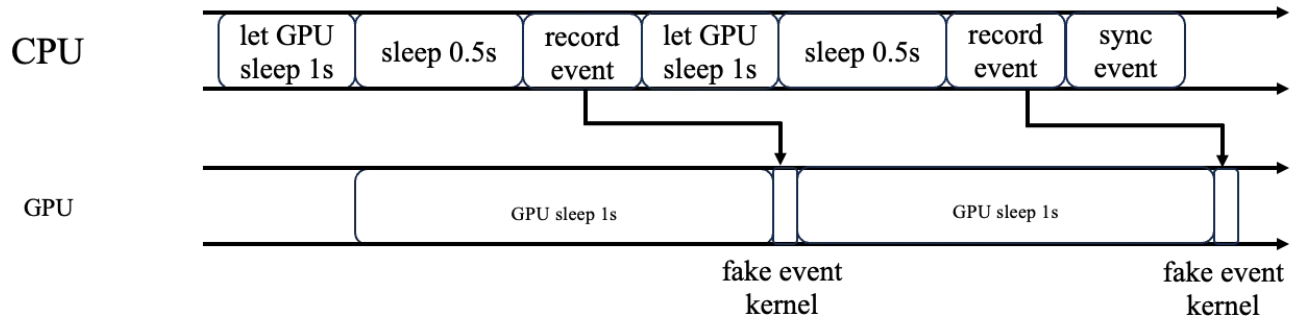
- 在 stream 2 上面执行 kernel 2，但是必须等到 stream 1 上面的 kernel 1 执行结束之后才能开始

CUDA event 同步 stream 依赖 C++

```cpp
// 在 stream1 上执行 kernel1
kernel1<<<gridDim, blockDim, 0, stream1>>>(d_data1, repeat);
cudaEventRecord(event1, stream1);

// stream2 等待 event1，并执行 kernel2
cudaStreamWaitEvent(stream2, event1, 0);
kernel2<<<gridDim, blockDim, 0, stream2>>>(d_data1, repeat);

// 在 stream1 上执行 kernel3
kernel3<<<gridDim, blockDim, 0, stream1>>>(d_data2, repeat);

// synchronize streams
cudaStreamSynchronize(stream1);
cudaStreamSynchronize(stream2);
```

CUDA Event 用于计时

除了用于 stream 之间的同步，另一个常见用法是使用 CUDA Event 来测量 GPU kernel 执行时间。`cudaEventRecord` 相当于把一个 event 插入到 stream 的队列里（fake event kernel），记录这个 event 执行的时刻。



计时 Bash

```bash
$./build/csrc/cuda_stream_event/event_elapsed_time 0.1 0.1
Elapsed time: 1048.58 milliseconds
[yiyin.zjh@gpu-public011167035133.cm10 ~/llm-inference-practice]
$./build/csrc/cuda_stream_event/event_elapsed_time 3 0.1
Elapsed time: 1048.1 milliseconds
[yiyin.zjh@gpu-public011167035133.cm10 ~/llm-inference-practice]
$./build/csrc/cuda_stream_event/event_elapsed_time 0.1 1.5
Elapsed time: 1048.58 milliseconds
[yiyin.zjh@gpu-public011167035133.cm10 ~/llm-inference-practice]
$./build/csrc/cuda_stream_event/event_elapsed_time 0.1 3
Elapsed time: 2052.15 milliseconds
[yiyin.zjh@gpu-public011167035133.cm10 ~/llm-inference-practice]
$./build/csrc/cuda_stream_event/event_elapsed_time 1.0 1.0
Elapsed time: 1048.58 milliseconds
[yiyin.zjh@gpu-public011167035133.cm10 ~/llm-inference-practice]
$./build/csrc/cuda_stream_event/event_elapsed_time 1.0 1.1
Elapsed time: 1051.98 milliseconds
```

此处为语雀内容卡片，点击链接查看：
https://aliyuque.antfin.com/yiyin.zjh/gg9fan/swh7zcdfqtcvgo3a?view=doc_embed

跨 GPU、跨进程的 stream 同步

`CUDA Stream` 以及 `CUDA Event` 不局限于单个 GPU、单个进程。stream 可以是来自不同 GPU 的 stream，甚至可以是来自不同进程之间的 stream (不同进程之间的 event 需要通过 `ipc` 传递)。

跨进程 stream 同步 Python

```python
def producer(i):
    dist.init_process_group(
        backend='nccl',
        init_method="tcp://127.0.0.1:39031",
        world_size=2,
        rank=0,
    )
    # 准备数据
    data = torch.randn((1024 * 1024, ), device=f"cuda:{i}")
    func, args = torch.multiprocessing.reductions.reduce_tensor(data)
    args = list(args)
    dist.broadcast_object_list([(func, args)], src=0)
    event = torch.cuda.Event(interprocess=True)
    # 广播 event 的 ipc_handle
    dist.broadcast_object_list([event.ipc_handle()], src=0)
    # 用一个 kernel 来处理数据
    for i in range(100):
        data += 1
    # 记录事件
    event.record()
    # 进程同步，确保 event wait 在 event record 之后
    dist.barrier()
    torch.cuda.synchronize()
    dist.destroy_process_group()


def consumer(j):
    dist.init_process_group(
        backend='nccl',
        init_method="tcp://127.0.0.1:39031",
        world_size=2,
        rank=1,
    )
    torch.cuda.set_device(j)
    # 接收数据
    recv = [None]
    dist.broadcast_object_list(recv, src=0)
    func, args = recv[0]
    args[6] = j
    data = func(*args)
    # 接收 event
    recv = [None]
    dist.broadcast_object_list(recv, src=0)
    event_handle = recv[0]
```

```
    event = torch.cuda.Event.from_ipc_handle(device=j, handle=event_handle)
    # 进程同步，确保 event wait 在 event record 之后
    dist.barrier()
    # 等待 event
    event.wait()
    # 判断数据正确性
    assert any([val == 100 for val in data]), "Data is not correct"
    dist.destroy_process_group()


if __name__ == "__main__":
    pi = mp.Process(target=producer, args=(0,))
    pj = mp.Process(target=consumer, args=(1,))
    pi.start()
    pj.start()
    pi.join()
    pj.join()
    assert pi.exitcode == 0 and pj.exitcode == 0, "Process failed"
```

关于 stream 的使用

stream 的概念非常清晰，使用起来也非常简单易懂。如果世界上每个使用 CUDA 的人，在调用每个 CUDA 函数的时候，都写明自己要用到哪个 stream，那么世界将变得非常美好。作为一位现代 CUDA 使用者，应该尽可能做到：

- kernel launch 的时候设计好 stream 参数，这样可以更好地利用 GPU 的硬件资源，做到多个 kernel并发执行。
- 尽可能使用带 stream 参数版本的显存相关函数，例如 cudaMallocAsync。

可惜，现实并不是这样。即使我们自己做到了这一点，也难免会遇到其它人的代码不符合这些要求：

- 很多人调用 cuda kernel 的时候，简单粗暴地直接 `kernel<<<gridDim, blockDim>>>()`，不指定 stream 参数。
- 有一些 cuda 函数，本身也没有 stream 参数。

此时，它们的 stream 参数，就是 default stream。而这个 default stream，是可以在编译期间控制的，详见：

此处为语雀内容卡片，点击链接查看：
https://aliyuque.antfin.com/yiyin.zjh/gg9fan/xolv45v2du5q9ya1?view=doc_embed
。其中有个 legacy default stream (也叫 null stream) 与所有 blocking stream 强依赖，即：

- null stream的任务完成之前，所有stream都得等着
- null stream加入任务之前，也得等所有的stream里面的任务都完成

为了解决这个问题，最简单的做法就是，创建 stream 的时候，尽可能用 non-blocking stream，这样的 stream 与 null stream 之间就没有依赖关系了。

在 PyTorch 的代码里，我们可以看到：

Pytorch default stream C++

```cpp
// ...
static constexpr unsigned int kDefaultFlags = cudaStreamNonBlocking;
// ...
C10_CUDA_CHECK(cudaStreamCreateWithPriority(&stream, kDefaultFlags, pri));
// ...
```

PyTorch 的 stream 都是用功能最丰富的 `cudaStreamCreateWithPriority` 函数创建的，而且默认的 flag 都是 `cudaStreamNonBlocking`，从而避免了与 null stream 之间的隐式依赖。当然，如果不创建 stream，那么 PyTorch 依然会使用 default stream。

编译命令

```
cmake -B build && cmake --build build
```

```
cmake -B build -DUSE_PER_THREAD_STREAM=ON -DCMAKE_PREFIX_PATH=`python -c 'import torch;print(torch.utils.cmake_prefix_path)'` && cmake --build build
```

Reference

一文读懂cuda stream与cuda event

llm-inference-practice/csrc/cuda_stream_event at main · yykzjh/llm-inference-practice

llm-inference-practice/python/cuda_stream_event at main · yykzjh/llm-inference-practice